# Cucumber Testing Report

## *Bell Pepper Monitor*

Web UI Test Automation

JavaScript | Cucumber.js | Playwright

**Name:** [PLACEHOLDER — fill in manually]

**Training ID:** [PLACEHOLDER — fill in manually]

**Role:** QA Intern / Software Quality Assurance

**Date:** [PLACEHOLDER — fill in manually]

Login workflow validation and failure evidence reporting

Project inspection: 21 October 2026
Execution evidence: 20 October 2026

# Table of Contents

1 Introduction ..... 3
2 Objectives ..... 3
3 Tools and Technologies ..... 4
4 Advantages of Cucumber and Playwright ..... 4
5 Project Setup ..... 5
6 Project Structure ..... 6
7 Getting Started ..... 7
8 Feature Automation ..... 10
9 Test Execution Results and Outputs ..... 18
10 Challenges Faced and Solutions ..... 21
11 Limitations and Disadvantages ..... 22
12 Conclusion ..... 22
Appendix Evidence and Technical References ..... 23

The contents list links to the main report sections. Section 8 includes a separate scenario index.

### Report scope

This report describes the implemented login UI automation and its supporting Cucumber reporting workflow. The latest saved execution records six functional passes and one deliberate diagnostic failure. The failed diagnostic verifies failure evidence capture; it is not an application defect or a functional pass.

Numbered screenshot placeholders identify evidence to insert manually. No application screenshots have been inserted. The cover details remain editable for the author.

# 1 Introduction

Bell Pepper Monitor is a React web application with a login page and a protected monitoring dashboard. The automation examined in this report validates the login interface, browser form constraints and navigation into the dashboard. It does not validate the monitoring data, sensor hardware, database correctness or every application route.

Behavior Driven Development (BDD) describes expected behaviour through examples that developers, testers and stakeholders can review. Cucumber.js executes those examples. Gherkin supplies the Feature, Scenario, Given, When, Then and And keywords used in features/dashboard.feature. Each executable step is matched to a JavaScript step definition. [R1]

Playwright supplies Chromium control, page interactions, locators, web assertions and screenshots. Cucumber.js is the test runner in this repository. Although chromium and expect are imported from @playwright/test, the suite is launched with cucumber-js rather than the Playwright Test runner. [R2; P1–P4]

**Verified outcome.** The saved HTML report contains seven executed scenarios: six functional scenarios passed and the temporary screenshot-verification scenario failed intentionally. All seven After hooks passed. These are historical results from 20 October 2026, inspected on 21 October 2026; no fresh browser run was performed for this report. [E1]

**Authentication boundary.** Login.jsx calls login with the email and name, and AuthContext.jsx stores a user object in React state. The password is not checked against an authentication service. A successful login test establishes the demonstrated UI flow and route transition only; it does not establish production authentication security. [P5]

# 2 Objectives

- Describe and execute readable examples for opening the login page, demo login and browser validation.
- Check required input behaviour, the default password input type and the login button state.
- Reuse email, password and submit actions through the existing login helper functions.
- Keep each scenario’s browser and page separate through its Cucumber World, then close them in the After hook.
- Produce an HTML execution report and attach an actual page screenshot when a scenario fails.
- Separate functional test outcomes from the temporary failure-capture diagnostic and retain traceable evidence.

# 3 Tools and Technologies

| Technology | Role in this project | Verified detail |
|---|---|---|
| JavaScript | Step definitions and helpers; ES module imports | type: module |
| Cucumber.js | Gherkin execution, hooks and HTML formatter | 13.3.0 installed |
| Playwright | Chromium control and expect assertions | 1.64.1 installed |
| Gherkin | Readable scenario specifications | 1 feature file |
| Node.js | Runs the JavaScript automation | 22.15.0 in saved run |
| npm | Dependency installation and script entry points | package-lock.json |
| Chromium | Browser launched by the test steps | headed mode |
| React and Vite | Application UI and development server | 19.2.8 / 8.2.2 installed |
| Visual Studio Code | Requested working IDE for source and terminals | IDE version not recorded |
| Node built-ins | Assertions, files, paths and unique screenshot names | assert, fs, path, crypto |

package.json declares @cucumber/cucumber ^13.3.0, @playwright/test ^1.64.1 and vite ^8.2.0. Installed package metadata was checked separately; declared version ranges are not exact installed versions. No npm version or Chromium build is recorded in the saved execution. [P1; E1]

# 4 Advantages of Cucumber and Playwright

**Readable specifications.** Gherkin makes actions and expected behaviour reviewable without reading every browser API call. The value depends on keeping step wording aligned with implementation. [R1]

**Reusable actions.** enterLoginDetails and clickLoginButton centralise repeated form operations. Changes to these selectors can be made in a shared helper, while assertions remain visible in the step definitions. [P3–P4]

**Focused browser checks.** Role and ID locators, visible-state waits and expect assertions make the tested conditions explicit. They support these UI checks but do not eliminate every source of timing or environment failure. [P3]

**Inspectable failure evidence.** The After hook saves a full-page PNG and attaches the same bytes to the HTML report. Scenario state is retained through capture and cleared after browser closure. [P3; E1]

# 5 Project Setup

The commands below reproduce the existing repository setup. They document how to prepare and run the suite; dependency installation and browser execution were not repeated while preparing this report. Run them from the project root in a suitable Node.js environment. [P1–P3; P6]

## 5.1 Prerequisites

Use Node.js compatible with the installed packages. The installed Cucumber.js 13.3.0 metadata accepts Node 22, 24 or 26 and above; Vite 8.2.2 requires Node ^20.19.0 or >=22.12.0. Node 22.15.0 is recorded in the saved test run. Chromium must be installed for the Playwright package, and a graphical desktop is required by the current headed launch configuration.

## 5.2 Dependencies and browser

```
npm install
npx playwright install chromium
```

The automation dependencies already appear under devDependencies in package.json. There is no need to add a different runner or rebuild the framework. package-lock.json records the resolved dependency graph. npm ci can be used for a clean lockfile-based environment, but it replaces node_modules and is not necessary for report preparation.

## 5.3 Start the application

```
# Terminal 1
npm run dev -- --port 5174 --strictPort

# Terminal 2
$env:BASE_URL = 'http://localhost:5174/'
npm test
```

Keep the Vite server running. --strictPort prevents a silent switch to a different port. The step definition defaults to http://localhost:5174/ and derives /login and / from BASE_URL. vite.config.js has no explicit server port. README.md also mentions 5173; align the active server URL with the test configuration. [P2; P6]

**[SCREENSHOT PLACEHOLDER — FIGURE 01]**

**Title: Package dependencies and scripts**

Insert: Capture package.json in Visual Studio Code, showing pretest, test and dev plus @cucumber/cucumber and @playwright/test.

Purpose: Shows the actual npm entry points and automation dependency declarations.

# 6 Project Structure

The tree below shows the existing automation files and relevant application context. Unrelated assets, hardware code, dependency contents and Git internals are omitted for readability. The documentation folder contains the deliverables added with this report. [P1–P6]

```
bell-pepper-monitor/
  features/
    dashboard.feature
  tests/
    helpers/
      login.helper.js
    step-definitions/
      dashboard.steps.js
  reports/
    cucumber-report.html
    screenshots/
      Temporary-test-to-verify-failure-screenshot-...png
  src/
    pages/Login.jsx
    context/AuthContext.jsx
    App.jsx
  documentation/cucumber-report/
  cucumber.cjs
  package.json
  package-lock.json
  vite.config.js
  README.md
```

dashboard.feature contains the feature named Bell Pepper Monitor Login. Its filename does not imply dashboard-module coverage. dashboard.steps.js contains step definitions, browser initialisation, assertions, the After hook and the intentional failing assertion. There is no separate hooks file, custom World class or Playwright configuration file in the inspected automation.

login.helper.js contains two reusable functions. cucumber.cjs selects feature and JavaScript paths and configures the terminal and HTML formatters. reports/screenshots stores actual failure PNGs; passing scenarios do not generate screenshots. The report directory is created by both the npm pretest script and cucumber.cjs.

**[SCREENSHOT PLACEHOLDER — FIGURE 02]**

**Title: Project folder structure**

Insert: Capture the Visual Studio Code Explorer with features, tests, reports, cucumber.cjs and package.json expanded.

Purpose: Demonstrates the actual organisation of the automation framework.

# 7 Getting Started

## 7.1 Cucumber configuration

cucumber.cjs uses CommonJS configuration in an application whose JavaScript files are ES modules. The default profile imports the step file and uses Cucumber's built-in progress and HTML formatters. The following configuration is taken from the project. [P2]

```
// Also prepare the report directory when running cucumber-js directly.
require('node:fs').mkdirSync('reports', { recursive: true });

module.exports = {
  default: {
    paths: ['features/**/*.feature'],
    import: ['tests/step-definitions/**/*.js'],
    format: [
      'progress',
      'html:reports/cucumber-report.html'
    ]
  }
};
```

**[SCREENSHOT PLACEHOLDER — FIGURE 03]**

**Title: Cucumber configuration**

Insert: Capture cucumber.cjs in Visual Studio Code, including the directory creation, paths, import and format entries.

Purpose: Shows which files execute and where the HTML report is written.

## 7.2 Playwright browser initialisation

The Given step, rather than a Before hook, launches a fresh Chromium browser for every scenario. It stores browser and page on the scenario World using a function callback. The launch uses headless: false and slowMo: 700; setDefaultTimeout(15000) applies to Cucumber steps. [P3]

```
this.browser = await chromium.launch({
  headless: false,
  slowMo: 700
});
this.page = await this.browser.newPage();
await this.page.goto(loginURL);
```

**[SCREENSHOT PLACEHOLDER — FIGURE 04]**

**Title: Playwright browser initialisation**

Insert: Capture the Given step and BASE_URL definitions in tests/step-definitions/dashboard.steps.js in Visual Studio Code.

Purpose: Demonstrates scenario browser creation and navigation to /login.

## 7.3 Reusable login helper

The helper file exposes named async functions for filling the two login fields and clicking submit. It accepts the page explicitly; it does not authenticate a real account, retain a session, launch a browser or wait for the dashboard. [P4]

```
export async function enterLoginDetails(page, email, password) {
  await page.locator('#auth-email').fill(email);
  await page.locator('#auth-password').fill(password);
}

export async function clickLoginButton(page) {
  await page.locator('#auth-submit-btn').click();
}
```

**[SCREENSHOT PLACEHOLDER — FIGURE 05]**

**Title: Reusable login helper**

Insert: Capture tests/helpers/login.helper.js in Visual Studio Code with both exported functions visible.

Purpose: Shows shared form actions and the actual ID selectors.

## 7.4 Step definitions and assertions

The step definitions import Given, When, Then, After, Status and setDefaultTimeout from @cucumber/cucumber, plus chromium and expect from @playwright/test. Node assert provides strict equality assertions. Matching step text connects the feature to those callbacks. [P3]

```
When('the user enters valid email and password', async function () {
  // Demo mode: this checks the UI flow, not real authentication security.
  await enterLoginDetails(this.page, 'qa@example.com', 'Test123!');
});
```

The redirect assertion waits for dashboardURL and checks pathname /. Validation uses form.noValidate, form.checkValidity(), email focus and the login URL. Required fields are checked through required and validity.valueMissing. Masking checks type=password; button checks cover visibility, enabled state and Log in text.

**[SCREENSHOT PLACEHOLDER — FIGURE 06]**

**Title: Step definitions and assertions**

Insert: Capture the validation and redirect assertions in tests/step-definitions/dashboard.steps.js in Visual Studio Code.

Purpose: Connects the Gherkin expectations to the implemented browser checks.

## 7.5 After hook and failure screenshots

After runs following every scenario. It identifies a failed result with result?.status === Status.FAILED and attempts capture only if this.page exists and is open. This order preserves the page for evidence before cleanup. [P3]

Selected lines from the existing failure branch are reproduced below; the surrounding condition and error handling are described in the following paragraphs.

```
const directory = path.resolve('reports', 'screenshots');
await mkdir(directory, { recursive: true });
const scenarioName = (pickle?.name || 'scenario')
  .replace(/[^a-zA-Z0-9_-]+/g, '-')
  .slice(0, 80);
const filename = `${scenarioName}-${Date.now()}-${randomUUID().slice(0, 8)}.png`;
const screenshot = await this.page.screenshot({
  path: path.join(directory, filename),
  fullPage: true,
  timeout: 5000
});
await this.attach(screenshot, 'image/png');
```

Names are sanitised and limited to 80 characters before the timestamp and eight-character UUID suffix are added. This avoids Windows-invalid name characters and reduces collisions. The screenshot timeout is 5 seconds. Capture or attachment exceptions are logged without replacing the original scenario failure.

The hook then waits 3 seconds if the page is still available, closes this.browser and resets this.browser and this.page to undefined in finally. A browser-close error is rethrown for a previously non-failed scenario; on an already failed scenario it is logged so the original failure remains primary.

If Chromium cannot launch, or the page has already closed, the hook cannot create a page screenshot. A passing hook alone does not prove a screenshot exists. Here, the diagnostic report attachment and matching PNG provide direct evidence that capture and attachment succeeded. [E1]

**[SCREENSHOT PLACEHOLDER — FIGURE 07]**

**Title: After hook and screenshot capture**

Insert: Capture the After hook in tests/step-definitions/dashboard.steps.js in Visual Studio Code, including the failed guard, screenshot call, attachment and browser cleanup.

Purpose: Demonstrates capture before teardown and error handling that preserves the original failure.

### 7.6 HTML report generation

npm test invokes cucumber-js after the pretest script creates reports. Loading cucumber.cjs also creates that directory for direct Cucumber invocation. The html:reports/cucumber-report.html formatter writes a self-contained report containing scenario and step statuses, errors and attached media. No separate HTML-report generator package is configured. [P1–P2]

A normal run replaces the configured HTML output. Existing failure PNGs remain in reports/screenshots. Preserve the HTML and relevant PNGs under a new evidence name before rerunning if the historical record is required. This report's result tables refer to the saved run identified in Section 9.

## 8 Feature Automation

All seven scenarios are in features/dashboard.feature under Feature: Bell Pepper Monitor Login. There are no scenario outlines, examples tables or tags in this feature. Each scenario begins by opening /login in a new browser. The six functional scenarios contain 19 Gherkin steps; the diagnostic contributes two more. [P2–P3]

| ID | Exact scenario name | Classification |
|---|---|---|
| TC01 | Open the monitoring application | Functional |
| TC02 | Login with valid details | Functional |
| TC03 | Login with invalid email format | Functional |
| TC04 | Empty Login Form Validation | Functional |
| TC05 | Password Field Masking | Functional |
| TC06 | Login Button Verification | Functional |
| DG01 | Temporary test to verify failure screenshot | Diagnostic only |

**[SCREENSHOT PLACEHOLDER — FIGURE 08]**

**Title: Feature file scenarios**

Insert: Capture features/dashboard.feature in Visual Studio Code with the feature name and scenario declarations visible; clearly identify the temporary diagnostic.

Purpose: Shows the exact implemented Gherkin scope without implying wider dashboard coverage.

## 8.1 Open the monitoring application

**Test case.** TC01 | Functional UI scenario

**Objective.** Confirm the login page and its primary controls are present.

**Preconditions.** The application is reachable at the configured URL. The fresh scenario has no in-memory logged-in user.

### Gherkin test steps

```
Scenario: Open the monitoring application
    Given the user opens the monitoring application
    Then the application page should be visible
```

**Expected result.** The Bell Pepper Monitor heading and email, password and submit controls are visible; the submit text is Log in.

**Verified actual result.** The visible-state waits and submit-text equality assertion completed successfully. The saved run records both Gherkin steps as passed.

**Status.** PASSED in the saved execution of 20 October 2026. [E1]

**Coverage boundary.** The scenario navigates directly to /login. It does not assert a redirect from the unauthenticated root route.

**[SCREENSHOT PLACEHOLDER — FIGURE 09]**
**Title: Open the monitoring application**
Insert: Capture the Chromium /login page with the heading and all login controls visible.
Purpose: Provides visual context for the login-page presence checks.

## 8.2 Login with valid details

**Test case.** TC02 | Functional UI scenario

**Objective.** Verify the demo login form can navigate to the dashboard with the supplied syntactically valid test data.

**Preconditions.** The server is available and the fresh browser is on /login. Test data: qa@example.com and Test123!. These are demo inputs, not verified production account credentials.

### Gherkin test steps

```
Scenario: Login with valid details
    Given the user opens the monitoring application
    When the user enters valid email and password
    And the user clicks the login button
    Then the user should be redirected to the dashboard
```

**Expected result.** After submit, the page reaches dashboardURL derived from BASE_URL; the pathname is /.

**Verified actual result.** The helper filled both inputs, clicked submit and the URL assertions passed. The saved run records all four Gherkin steps as passed.

**Status.** PASSED in the saved execution of 20 October 2026. [E1]

**Coverage boundary.** The assertion checks navigation, not dashboard widgets, sensor values, backend authorisation or password validation.

**[SCREENSHOT PLACEHOLDER — FIGURE 10]**
**Title: Login with valid details**
Insert: Capture Chromium after demo login with the address bar showing / and the dashboard visible.
Purpose: Illustrates the tested route transition after the demo form submission.

## 8.3 Login with invalid email format

**Test case.** TC03 | Functional UI scenario

**Objective.** Verify native browser validation prevents submission of a malformed email.

**Preconditions.** The fresh browser is on /login. The helper uses invalid-email and Test123!. The form has not disabled native validation.

### Gherkin test steps

```
Scenario: Login with invalid email format
    Given the user opens the monitoring application
    When the user enters an invalid email format
    And the user clicks the login button
    Then the browser should prevent form submission
```

**Expected result.** form.noValidate is false, form.checkValidity() is false, #auth-email is focused and the URL remains loginURL.

**Verified actual result.** All four validation conditions were satisfied. The saved run records all four Gherkin steps as passed.

**Status.** PASSED in the saved execution of 20 October 2026. [E1]

**Coverage boundary.** The test does not compare validation-message wording or assert an application-generated error. This is format validation, not server-side credential rejection.

**[SCREENSHOT PLACEHOLDER — FIGURE 11]**

**Title: Login with invalid email format**

Insert: Capture Chromium after submitting invalid-email, with the invalid email field and any native validation prompt visible.

Purpose: Shows the browser-level rejection state for malformed email input.

## 8.4 Empty Login Form Validation

**Test case.** TC04 | Functional UI scenario

**Objective.** Verify the browser blocks empty submission and both login inputs are required.

**Preconditions.** The fresh browser is on /login. The helper fills both inputs with empty strings before submit.

### Gherkin test steps

```
Scenario: Empty Login Form Validation
    Given the user opens the monitoring application
    When the user leaves email and password empty
    And the user clicks the login button
    Then the browser should prevent form submission
    And both login fields should be required
```

**Expected result.** Native validation remains enabled, form validity is false, email receives focus and /login remains active. Both inputs have empty values, required=true and validity.valueMissing=true.

**Verified actual result.** The invalid-form, focus, URL, empty-value and required-field assertions completed successfully. The saved run records all five Gherkin steps as passed.

**Status.** PASSED in the saved execution of 20 October 2026. [E1]

**Coverage boundary.** A screenshot alone cannot demonstrate both DOM validity flags. Use the step-definition evidence alongside the UI capture.

**[SCREENSHOT PLACEHOLDER — FIGURE 12]**
**Title: Empty Login Form Validation**
Insert: Capture Chromium after empty submission with both empty fields and the first invalid field visible.
Purpose: Provides visual context for the required-field validation checks.

## 8.5 Password Field Masking

**Test case.** TC05 | Functional UI scenario

**Objective.** Check that the password field uses a password input type by default.

**Preconditions.** The fresh browser is on /login and the password-visibility toggle has not been activated.

### Gherkin test steps

```
Scenario: Password Field Masking
    Given the user opens the monitoring application
    Then the password field should be masked
```

**Expected result.** #auth-password has type=password.

**Verified actual result.** The Playwright attribute assertion passed. The saved run records both Gherkin steps as passed.

**Status.** PASSED in the saved execution of 20 October 2026. [E1]

**Coverage boundary.** This scenario does not enter a password, test the eye toggle, inspect visual masking characters or establish encryption and credential-storage security.

**[SCREENSHOT PLACEHOLDER — FIGURE 13]**

**Title: Password Field Masking**

Insert: Capture Chromium /login with DevTools Elements showing #auth-password and type="password"; the field may remain empty.

Purpose: Demonstrates the exact DOM property checked by the automated scenario.

## 8.6 Login Button Verification

**Test case.** TC06 | Functional UI scenario

**Objective.** Check that the default login submit control is visible, enabled and labelled correctly.

**Preconditions.** The fresh browser is on /login with the default login tab active. Email and password may be empty.

### Gherkin test steps

```
Scenario: Login Button Verification
    Given the user opens the monitoring application
    Then the login button should be visible and enabled
```

**Expected result.** #auth-submit-btn is visible and enabled and its text is Log in.

**Verified actual result.** All three expect assertions passed. The saved run records both Gherkin steps as passed.

**Status.** PASSED in the saved execution of 20 October 2026. [E1]

**Coverage boundary.** The scenario does not click submit or verify loading, disabled-during-submit or keyboard activation behaviour. Required-field validation is covered by TC04.

**[SCREENSHOT PLACEHOLDER — FIGURE 14]**

**Title: Login Button Verification**

Insert: Capture Chromium /login with the Log in button visible; include DevTools if documenting enabled state.

Purpose: Shows the submit control whose visibility, state and text are asserted.

## 8.7 Temporary test to verify failure screenshot

**Diagnostic case.** DG01 | Temporary framework diagnostic; excluded from normal functional coverage.

**Objective.** Exercise the existing failure branch and verify that a live page screenshot is saved and attached before the browser closes.

**Preconditions.** The application is reachable and Chromium can open /login. The temporary scenario and its deliberately failing Then definition are present in the current files.

#### Gherkin test steps

```
Scenario: Temporary test to verify failure screenshot
    Given the user opens the monitoring application
    Then a deliberately failing test should fail
```

**Diagnostic trigger.** The Then definition executes assert.equal(true, false, 'Intentional failure to test screenshot capture'). This assertion is expected to fail. [P3]

**Expected result.** Cucumber marks the scenario FAILED; the After hook captures and attaches a PNG if the page is available, then closes the browser. The diagnostic is never counted as a normal functional pass.

**Verified actual result.** The opening step passed and the deliberate assertion failed with the expected message. The After hook passed. The HTML contains one image/png attachment of 33,674 decoded bytes; its SHA-256 hash matches the existing PNG in reports/screenshots. [E1]

**Status.** FAILED intentionally. Failure capture and attachment were verified as diagnostic outcomes. The overall saved run reports success=false.

The diagnostic is currently untagged and is selected by the default feature glob. A normal npm test therefore includes it. Section 9 documents how to select only the six functional scenarios without changing the feature file.

**[SCREENSHOT PLACEHOLDER — FIGURE 15]**

**Title: Diagnostic failure attachment**

Insert: Open reports/cucumber-report.html in a browser, expand the temporary scenario and capture the deliberate error and attached image together.

Purpose: Shows that the failed diagnostic retained the expected error and a real attachment.

# 9 Test Execution Results and Outputs

## 9.1 Execute and generate reports

Run the application and automation from separate terminals as described in Section 5. npm test executes every current scenario, including the intentional diagnostic. A nonzero exit is expected while that diagnostic remains included; no fresh exit code was observed during report preparation. [P1–P3]

```
# Run the current complete feature
npm test

# Execute only functional scenarios without editing test files
npm test -- --name '^(?!Temporary test to verify failure screenshot$).*'

# Check matching without browser execution
npm test -- --dry-run

# Execute only the screenshot diagnostic
npm test -- --name '^Temporary test to verify failure screenshot$'
```

These are reproduction commands, not evidence of additional completed runs. The functional-only expression excludes the exact diagnostic name. A dry run checks step matching; it does not validate UI behaviour or create a real pass result. The diagnostic-only command intentionally fails when its assertion is reached.

## 9.2 Preserve and inspect evidence

The configured HTML destination is reports/cucumber-report.html. Open it in a browser after a completed run. Before running any command above, preserve existing report evidence using a new, unused filename: each invocation can replace the configured HTML, including filtered or dry runs. Screenshot files from earlier failures are not automatically removed. [P2; P6]

The existing historical report was preserved during this documentation work. No test logic, application source or existing output was altered. The separate evidence-summary.json records source hashes and the saved execution details for traceability.

**[SCREENSHOT PLACEHOLDER — FIGURE 16]**

**Title: Successful functional execution in the terminal**

Insert: In a VS Code terminal, capture an actual completed functional-only run showing six scenarios passed. This terminal screenshot is not currently available; capture it only after performing a new run.

Purpose: Provides optional new terminal evidence for the functional subset. It must be dated separately from the saved run analysed here.

## 9.3 Latest available execution summary

Evidence source: embedded Cucumber Messages in the existing reports/cucumber-report.html. Its feature source matches the current dashboard.feature after normalising line endings and surrounding whitespace. Scenario results were reconstructed from test cases, step-finished events and hook events. Terminal output was not retained, so the values below are not presented as a copied terminal transcript. [E1]

| Measure | Verified value |
|---|---|
| Run start in Sri Lanka time | 20 October 2026 22:57:16.515 |
| Run finish in Sri Lanka time | 20 October 2026 22:58:12.569 |
| Recorded run elapsed time | 56.054 seconds |
| Total scenarios | 7 |
| Passed scenarios | 6 functional |
| Failed scenarios | 1 intentional diagnostic |
| Skipped or undefined scenarios | 0 / 0 |
| Gherkin steps | 21 total: 20 passed, 1 failed |
| Skipped or undefined Gherkin steps | 0 / 0 |
| After hooks | 7 total, all passed |
| Run success flag | false |
| Recorded environment | Windows x64; Node.js 22.15.0; Cucumber.js 13.3.0 |

The 56.054 seconds is the difference between the Cucumber testRunStarted and testRunFinished timestamps. It includes the framework's browser setup, slowed actions and teardown observation waits; it is not a measure of application response time. The seven After hooks are separate from the 21 Gherkin steps.

Functional subset: 6 scenarios passed with 19 steps passed and no failed functional step. Diagnostic subset: 1 scenario failed with 1 step passed and 1 deliberately failed. No retries are recorded. There is no separate functional-only execution artifact in the available evidence.

**[SCREENSHOT PLACEHOLDER — FIGURE 17]**

**Title: Final execution summary**

Insert: Open the saved reports/cucumber-report.html in a browser and capture its summary showing six passed and one failed scenario. Preserve the diagnostic classification in the caption.

Purpose: Supports the exact totals of the saved full run without claiming an all-green suite.

## 9.4 Scenario results and outputs

| Case | Steps passed / failed | Scenario status |
|---|---|---|
| TC01 Open application | 2 / 0 | PASSED |
| TC02 Demo login | 4 / 0 | PASSED |
| TC03 Invalid email format | 4 / 0 | PASSED |
| TC04 Empty form | 5 / 0 | PASSED |
| TC05 Password input type | 2 / 0 | PASSED |
| TC06 Login button | 2 / 0 | PASSED |
| DG01 Screenshot diagnostic | 1 / 1 | FAILED intentionally |

**HTML output.** The saved report provides feature and scenario details, step outcomes, the intentional assertion error and a PNG attachment. It is the execution evidence used for this report. The failed diagnostic remains visible as a failure in the HTML; the functional subset is interpreted separately. [E1]

**Saved PNG.** The matching file is in reports/screenshots and begins Temporary-test-to-verify-failure-screenshot-1792517288213-. Its full name and SHA-256 are recorded in evidence-summary.json. The PNG is 1280 × 720 pixels. Matching the file bytes to the embedded attachment verifies the saved-image and attachment relationship.

**[SCREENSHOT PLACEHOLDER — FIGURE 18]**

**Title: Intentional failed execution in the terminal**

Insert: Capture a new diagnostic-only run in a VS Code terminal showing the expected assertion message and failed count. No historical terminal capture is available.

Purpose: Documents the deliberate failure trigger; a future run is additional evidence, not the original saved execution.

**[SCREENSHOT PLACEHOLDER — FIGURE 19]**

**Title: Saved failure screenshot**

Insert: Open the existing PNG under reports/screenshots in VS Code or an image viewer and capture the image tab with its filename visible.

Purpose: Shows the actual saved diagnostic page screenshot and its storage location.

**[SCREENSHOT PLACEHOLDER — FIGURE 20]**

**Title: Generated Cucumber HTML report**

Insert: Open reports/cucumber-report.html in a browser and capture the feature details and expanded scenario results.

Purpose: Demonstrates the generated report layout and the link between readable scenarios and execution outcomes.

# 10 Challenges Faced and Solutions

The issues below are supported by the current code, README and saved diagnostic output. They describe implemented safeguards and observable configuration concerns; they are not an invented history of defects or debugging sessions.

## 10.1 Matching the application URL

The README includes a 5173 start URL while the current step file defaults to 5174. Vite has no fixed port in vite.config.js. The implemented BASE_URL override and URL-derived login and dashboard paths allow the suite to target the active server. Starting Vite with --port 5174 --strictPort gives a reproducible command for the current default. [P2–P3; P6]

## 10.2 Verifying browser validation precisely

Invalid email format and missing required fields are native browser conditions. The implementation checks noValidate, checkValidity(), focus, unchanged URL and the required/valueMissing flags, avoiding dependence on localised validation-message wording. This verifies DOM and navigation behaviour without claiming server-side validation. [P3; P5]

## 10.3 Capturing evidence before cleanup

Closing the browser before taking a screenshot would remove the live page. The implemented After hook captures first, attaches the returned bytes, then closes the browser. The saved diagnostic confirms that a PNG was both written and attached. When a page is unavailable, capture is skipped rather than fabricated. [P3; E1]

## 10.4 Preserving the original failure

Screenshot and attachment errors are caught and logged; browser-close errors on an already failed scenario are also logged. The original deliberate assertion remains visible in the saved report. Windows-safe names, timestamps and UUID suffixes provide a practical safeguard for storing multiple failure images. [P3; E1]

## 10.5 Keeping diagnostic outcomes distinct

The temporary assertion remains an untagged scenario in the normal feature. Its presence makes the saved full suite unsuccessful even though all six functional scenarios pass. The functional-only name filter documented in Section 9 separates routine execution without editing the existing suite. Tagging or moving diagnostics would be a future framework improvement.

# 11 Limitations and Disadvantages

**Demo authentication.** No password-to-account verification occurs in the examined login flow. These tests do not establish credential protection, server-side authorisation, secure sessions, lockout, rate limiting or production authentication security. The invalid-email scenario tests HTML format constraints, not an incorrect-password response. [P5]

**Limited functional scope.** Coverage consists of six login UI scenarios. Registration, logout, password-visibility toggling, recovery links, monitoring widgets, plant data, history, power, communication, settings and sensor data accuracy are not asserted by this feature. Dashboard redirection is checked by URL only. [P2–P5]

**Maintenance and readability.** UI or copy changes can require updates to ID locators, heading text and assertions. Gherkin remains useful only while step wording stays aligned with the code. Coding knowledge is still required to maintain browser interactions and hooks.

**Environment and duration.** Only Chromium is launched. The current suite is headed, uses slowMo: 700 and waits 3 seconds during teardown. Browser availability, the graphical session, server port and timeout configuration can affect execution. There is no implemented browser matrix, explicit retry configuration or CI workflow among the inspected test files.

**Evidence availability.** Passing scenarios have no automatic screenshots. Failure screenshots require an available live page. The report covers the saved execution of 20 October 2026, not a new run or a guarantee of future behaviour. Its HTML can be replaced by subsequent executions; old PNGs can outlive the report that created them.

**Testing boundaries.** Automated UI checks complement manual exploration, accessibility, API, integration, performance and security testing. None of those broader activities can be inferred from the seven scenarios or their saved results.

# 12 Conclusion

Bell Pepper Monitor has an implemented JavaScript BDD login suite using Cucumber.js as the runner and Playwright for Chromium interactions. The framework includes readable Gherkin scenarios, shared login helpers, per-scenario browser creation, teardown, HTML reporting and failure screenshots attached before cleanup.

The verified saved execution contains six passed functional scenarios and one intentional diagnostic failure. The diagnostic produced a matching saved PNG and report attachment. This supports the implemented failure-evidence workflow while leaving the full run correctly recorded as unsuccessful.

Future work can separate diagnostic scenarios from routine runs, add assertions for dashboard content, cover registration and logout, and introduce real authentication integration tests when that functionality exists. Browser-matrix execution and CI can be considered after the headed configuration and environment requirements are addressed. These are proposed improvements, not completed coverage.

# Appendix Evidence and Technical References

Project paths below are relative to the Bell Pepper Monitor root. Inspection date: 21 October 2026. The accompanying evidence-summary.json retains full SHA-256 hashes of the inspected sources and saved HTML, the execution identity, step results and the PNG match.

| ID | Source | Use in this report |
|---|---|---|
| P1 | package.json; package-lock.json; installed package metadata | Scripts, versions and Node requirements |
| P2 | cucumber.cjs; features/dashboard.feature | Discovery, formatters and exact scenarios |
| P3 | tests/step-definitions/dashboard.steps.js | Launch, steps, assertions, hook and diagnostic |
| P4 | tests/helpers/login.helper.js | Reusable input and submit functions |
| P5 | src/pages/Login.jsx; src/context/AuthContext.jsx; src/App.jsx | Demo authentication and routes |
| P6 | README.md; vite.config.js | Setup instructions and port configuration |
| E1 | reports/cucumber-report.html; reports/screenshots/*.png | Saved execution and attachment evidence |

## Execution identity

Cucumber run ID: 26a3a3a3-288e-4fb5-9ce2-687ae35b1806

UTC start: 2026-10-20T17:27:16.515Z
UTC finish: 2026-10-20T17:28:12.569Z

HTML SHA-256: f52a4d09609983194cc65a643fdc89148a32ff7bb291cd2e92a28046f8154bc6

PNG SHA-256: a466256ee301e557bb33996e77a1a80f961b422ceeca76ddeb7e09cf0bb88db5

## Technical references

R1 Cucumber Gherkin Reference. Keyword structure and step definitions. Accessed 21 October 2026. https://cucumber.io/docs/gherkin/reference/

R2 Playwright Library documentation. Browser automation and the distinction between library APIs and the Playwright Test runner. Accessed 21 October 2026. https://playwright.dev/docs/library

The supplied reference PDF was used for section order and presentation cues only. Its identity details, application content, automation code, screenshots and execution results are not part of this report.